

Player & Obstacle Class Refactor (Using Models)

Refactor my obstacle class to take in the model path instead of using verticies and indicies:

Obstacle.h

```
#pragma once

#include <string>

#include <glm/glm.hpp>

#include <glm/gtc/matrix_transform.hpp>

#include "Model.h"

class Obstacle
{
public:
    Obstacle(
        const std::string& modelPath,
        glm::vec3 position,
        float moveSpeed,
        float rotationAngle,
        float rotationSpeed,
        glm::vec3 rotationAxis,
        glm::vec3 scaleAmount
    );

    void draw(Shader& shader) const;

    void update(float deltaTime);

    glm::mat4 getModel() const;

private:
```

```
Model obstacleModel;

glm::vec3 position;
glm::vec3 scaleAmount;
glm::vec3 rotationAxis;

float moveSpeed;
float rotationAngle;
float rotationSpeed;
};
```

Obstacle.cpp

```
#include "Obstacle.h"

Obstacle::Obstacle(
    const std::string& modelPath,
    glm::vec3 position,
    float moveSpeed,
    float rotationAngle,
    float rotationSpeed,
    glm::vec3 rotationAxis,
    glm::vec3 scaleAmount
)
: obstacleModel(modelPath),
  position(position),
  moveSpeed(moveSpeed),
  rotationAngle(rotationAngle),
  rotationSpeed(rotationSpeed),
```

```

        rotationAxis(rotationAxis),
        scaleAmount(scaleAmount)
    {
    }

void Obstacle::draw(Shader& shader) const
{
    obstacleModel.Draw(shader);
}

void Obstacle::update(float deltaTime)
{
    position.z += moveSpeed * deltaTime;
    rotationAngle += rotationSpeed * deltaTime;
}

glm::mat4 Obstacle::getModel() const
{
    glm::mat4 model = glm::mat4(1.0f);

    model = glm::translate(model, position);
    model = glm::rotate(model, rotationAngle, rotationAxis);
    model = glm::scale(model, scaleAmount);

    return model;
}

```

*Updated to use “const std::string& modelPath” inside of index and verticies

Model Player Collision

I have updated my game to use imported obj models instead of drawn geometry, this means my player collision needs updated. Which type of collision would be best to measure the models bounds?

Oriented Bounding Box (OBB) — most accurate

If you want collision that **follows rotation**, you need OBBs, not AABBs:

- They use the model's basis vectors.
- They rotate with the object.
- They are much tighter than AABBs.

ASSIMP gives you the vertex data, so you can compute an OBB easily — but collision math is more complex.

If your game is simple, OBB might be overkill.

Method	Accurate?	Rotation Friendly?	Performance	Recommended For
AABB from OBJ	Medium	✗ No	Very fast	Simple models, non-rotating objects
Bounding Sphere	Good	✓ Yes	Very fast	Asteroids, ships, anything round-ish
OBB	Very high	✓ Yes	Medium	Complex shapes where gameplay depends on precision

OBB Implementation Improvements

*After updating player collision towards the new mesh system

Below is an updated version of my `playerObstacleCollision` using the mesh verticies, can you evaluate the efficiency of it:

🔍 What's good

- ✓ It *technically* builds an AABB from the OBJ mesh
- ✓ It accounts for **scale**
- ✓ It performs a simple overlap test
- ✓ It's understandable and debuggable

For a beginner collision system, this is perfectly functional.

✗ What is *not* good / what's wrong

1. ! You recompute the min/max of every vertex every frame

That's extremely inefficient. OBJ meshes easily have 500–10,000+ vertices.

You should compute the mesh-space AABB once when loading the model, not per-frame.

2. ! Not an OBB — this is still an AABB

Your code creates a world-space axis-aligned bounding box. Even if the asteroid rotates, your bounding box does **not** rotate.

So your collision shape is wrong if you expect OBB behavior.

3. ! You ignore the model matrix

The line:

```
float x = v.x * obstacleScale.x + obstaclePosition.x;
```

...does not include:

- rotation
- translation from the model matrix
- arbitrary transforms
- non-uniform scaling with rotation

Correct OBB requires applying the **full model matrix** to each vertex:

```
vec4 worldPos = obstacle.getModel() * vec4(v, 1.0f);
```

Your version only handles position + scale.

4. ! Player bounding box is guessed, not based on actual mesh

```
vec3 playerSize = player.getScale() * 0.1f;
```

This “0.1f” is arbitrary. If the player mesh is changed, your collision size becomes wrong.

5. ! Signature uses pass-by-value

```
bool playerObstacleCollision(Player player, Obstacle obstacle)
```

This copies the player and obstacle every frame, implicitly rebuilding model matrices.
Use references:

```
bool playerObstacleCollision(const Player& player, const Obstacle& obstacle)
```

✓ How to improve toward a proper OBB system

Step 1 — Precompute mesh-space AABB once

During the model loading process:

```
mesh.minBounds
```

```
mesh.maxBounds
```

These never change.

Step 2 — Convert this AABB into an OBB using the object's model matrix

An OBB consists of:

- center
- half extents
- orientation matrix (3×3)

You can compute it like:

```
glm::mat4 model = obstacle.getModel();
```

```
glm::vec3 center = (min + max) * 0.5f;
```

```
glm::vec3 halfSize = (max - min) * 0.5f;
```

```
glm::vec3 worldCenter = glm::vec3(model * glm::vec4(center, 1.0f));
```

```
glm::mat3 rotation = glm::mat3(model); // remove translation
```

This gives you the OBB in world space.

Step 3 — Use SAT (Separating Axis Theorem)

OBB–OBB uses 15 axes, but is still very fast.

Even simpler: player might be AABB / sphere, asteroid might be OBB.

 **If you want a minimal, clean improvement (still AABB but proper)**

Replace:

```
float x = v.x * obstacleScale.x + obstaclePosition.x;
```

with:

```
glm::vec3 world = glm::vec3(obstacle.getModel()) * glm::vec4(v, 1.0f);
```

Camera clamping and smoothing

Can you adjust my camera movement logic to apply the max angle to prevent the camera from being moved past the intended max angle. Can you then also apply a smoothed X and Y for when moving the mouse up to the window width and height:

```
float lastX = windowWidth / 2.0f;
```

```
float lastY = windowHeight / 2.0f;
```

```
double mouseX, mouseY;
```

```
glfwGetCursorPos(window, &mouseX, &mouseY);
```

```
float sensitivity = 0.1f;
```

```
float maxAngle = glm::radians(3.0f);
```

```
float deltaX = (float)(mouseX - lastX);
```

```
float deltaY = (float)(mouseY - lastY);
```

```
lastX = mouseX;
```

```
lastY = mouseY;
```

```
// Add senseitivity
```

```
float camX = deltaX * sensitivity;
```

```
float camY = deltaY * sensitivity;
```

```
mat4 cameraTilt = rotate(mat4(1.0f), camX, vec3(0, 1, 0));
```

```
cameraTilt = rotate(cameraTilt, camY, vec3(1, 0, 0));
```

```
view = cameraTilt * lookAt(cameraPosition, cameraPosition +  
cameraFront, cameraUp);
```

1. Normalize to range -1 → +1

This gives consistent proportional movement:

```
float nx = (mouseX / windowWidth) * 2.0f - 1.0f;
```

```
float ny = (mouseY / windowHeight) * 2.0f - 1.0f;
```

Now:

- mouse at left edge → nx = -1
- center → nx = 0
- right edge → nx = +1

Same for ny.

Much smoother and intuitive.

2. Apply tiny scaling

You want very subtle tilt, so scale the normalized value by a small angle:

```
float maxAngle = glm::radians(3.0f); // tiny tilt
```

```
float dx = nx * maxAngle;
```

```
float dy = ny * maxAngle;
```

Now even at screen edges, tilt never exceeds 3 degrees.

3. Smooth with interpolation

Keeps things polished:


```
static float smoothedX = 0.0f;
```

```
static float smoothedY = 0.0f;
```

```
float smooth = 0.12f;
```

```
smoothedX = glm::mix(smoothedX, dx, smooth);
```

```
smoothedY = glm::mix(smoothedY, dy, smooth);
```

Only calculating obstacle AABB once

I have updated my player collision so that it is calculated within the obstacle class, meaning the collision box isn't calculated every frame. However, currently it isn't calculating the coordinated correctly

```
// Collision detection
```

```
bool playerObstacleCollision(const Player& player, const Obstacle& obstacle)
```

```
{
```

```
    // Player boundin
```

```
    vec3 playerHalfSize = player.getScale() * 0.1f;
```

```
    vec3 playerPos = player.getPosition();
```

```
    // Get obstacle model
```

```
    mat4 model = obstacle.getModel();
```

```
    // Get obstacle AABB in world space
```

```
    vec3 minAABB, maxAABB;
```

```
    obstacle.getAABB(minAABB, maxAABB);
```

```
    bool playerCollideAABB =
```

```
        (playerPos.x + playerHalfSize.x > minAABB.x) &&
```

```
        (playerPos.x - playerHalfSize.x < maxAABB.x) &&
```

```
        (playerPos.y + playerHalfSize.y > minAABB.y) &&
```

```
(playerPos.y - playerHalfSize.y < maxAABB.y) &&  
(playerPos.z + playerHalfSize.z > minAABB.z) &&  
(playerPos.z - playerHalfSize.z < maxAABB.z);
```

```
//// Initialize min/max
```

```
//float minX = std::numeric_limits<float>::max();  
//float minY = std::numeric_limits<float>::max();  
//float minZ = std::numeric_limits<float>::max();  
//float maxX = -std::numeric_limits<float>::max();  
//float maxY = -std::numeric_limits<float>::max();  
//float maxZ = -std::numeric_limits<float>::max();
```

```
//// Iterate through all vertices of the obstacle model
```

```
//for (const auto& mesh : obstacle.obstacleModel.meshes)  
//{  
//    for (const auto& vertex : mesh.vertices)  
//    {  
//        vec4 w = model * vec4(vertex.Position, 1.0f);  
  
//        // Add to min/max  
//        minX = std::min(minX, w.x);  
//        minY = std::min(minY, w.y);  
//        minZ = std::min(minZ, w.z);  
//        maxX = std::max(maxX, w.x);  
//        maxY = std::max(maxY, w.y);  
//        maxZ = std::max(maxZ, w.z);  
//    }  
//}
```

```

        //// Collision check

        //bool playerCollide =

        //    (playerPos.x + playerHalfSize.x > minX) &&
        //    (playerPos.x - playerHalfSize.x < maxX) &&
        //    (playerPos.y + playerHalfSize.y > minY) &&
        //    (playerPos.y - playerHalfSize.y < maxY) &&
        //    (playerPos.z + playerHalfSize.z > minZ) &&
        //    (playerPos.z - playerHalfSize.z < maxZ);

        return playerCollideAABB;
    }

void Obstacle::createAABB()
{
    // Initialize min/max values

    modelMin = glm::vec3(std::numeric_limits<float>::max());
    modelMax = glm::vec3(std::numeric_limits<float>::lowest());

    // Iterate through all vertices of the obstacle model
    for (const auto& mesh : obstacleModel.meshes)
    {
        for (const auto& vertex : mesh.vertices)
        {
            // Add to min/max

            modelMin.x = std::min(modelMin.x, vertex.Position.x);
            modelMin.y = std::min(modelMin.y, vertex.Position.y);
            modelMin.z = std::min(modelMin.z, vertex.Position.z);

```

```

        modelMax.x = std::max(modelMax.x, vertex.Position.x);

        modelMax.y = std::max(modelMax.y, vertex.Position.y);

        modelMax.z = std::max(modelMax.z, vertex.Position.z);
    }
}
}

void Obstacle::getAABB(glm::vec3& minOut, glm::vec3& maxOut) const
{
    minOut = modelMin;

    maxOut = modelMax;
}

```

🔧 **Fix: Add a method to transform AABB → world space**

Put this inside Obstacle.cpp:

```

void Obstacle::getWorldAABB(glm::vec3& outMin, glm::vec3& outMax) const
{
    glm::mat4 modelMat = getModel();

    glm::vec3 minP(FLT_MAX);

    glm::vec3 maxP(-FLT_MAX);

    glm::vec3 corners[8] = {
        {modelMin.x, modelMin.y, modelMin.z},
        {modelMax.x, modelMin.y, modelMin.z},
        {modelMin.x, modelMax.y, modelMin.z},
        {modelMax.x, modelMax.y, modelMin.z},
        {modelMin.x, modelMin.y, modelMax.z},
        {modelMax.x, modelMin.y, modelMax.z},
        {modelMin.x, modelMax.y, modelMax.z},
        {modelMax.x, modelMax.y, modelMax.z},
    };
}

```

```

        {modelMin.x, modelMax.y, modelMax.z},
        {modelMax.x, modelMax.y, modelMax.z}
    };

    for (int i = 0; i < 8; i++)
    {
        glm::vec4 worldV = modelMat * glm::vec4(corners[i], 1.0f);
        glm::vec3 p(worldV);

        minP = glm::min(minP, p);
        maxP = glm::max(maxP, p);
    }

    outMin = minP;
    outMax = maxP;
}

```

Upgrade to add lighting

I currently use MVP in order to display models within my OpenGL project. The shaders load the relevant texture, or use a default colour if not available. Can you help me transition my shaders to use BlinnPhong to render lighting

1. Why your current shaders cannot support lighting

Right now your fragment shader only knows:

- Texture coordinates
- A color (or texture)

That's enough for *unlit rendering*, but **lighting requires geometry context**:

Lighting term Requires

Diffuse Surface normal + light direction

Specular Normal + view direction

Lighting term Requires

Blinn Normal + halfway vector

Attenuation Fragment position

So before we even talk about Blinn, your pipeline must provide:

- **World-space fragment position**
- **World-space surface normal**

This is the key architectural shift.

2. Expanding the vertex attributes (Normals are non-negotiable)

Your current vertex shader only consumes position and UVs. For lighting, you must also pass **normals**.

This means:

- Your mesh data must contain normals
- You bind them as a vertex attribute
- You pass them forward to the fragment shader

At the vertex shader level, add:

- aNormal input
- Normal output
- FragPos output

Why both?

- FragPos → lighting direction math
 - Normal → surface orientation
-

3. MVP → Model / View / Projection (this is not optional)

Lighting *cannot* be done correctly with a single MVP matrix.

Why?

- MVP collapses all coordinate spaces into clip space
- Lighting math must happen in **world space** (or view space)

So instead of:

```
uniform mat4 mvpln;
```

You must split:

- model
- view
- projection

This is not just stylistic — it is mathematically required.

4. Correct normal transformation (this is where many implementations are wrong)

This line matters more than people think:

```
Normal = mat3(transpose(inverse(model))) * aNormal;
```

Why not just `model * normal`?

Because:

- Non-uniform scaling breaks normals
- Inverse-transpose preserves perpendicularity

If you skip this, your lighting will look “almost right” until you scale something — then it breaks in subtle ways.

5. Passing world-space data to the fragment shader

At the end of the vertex shader, you now output:

- FragPos → world-space position
- Normal → corrected world-space normal
- TexCoords → unchanged

The fragment shader now has *everything* it needs to compute lighting.

6. Introducing Blinn-Phong (what changes vs Phong)

Classic Phong:

```
reflect(-L, N) · V
```

Blinn:

$N \cdot \text{normalize}(L + V)$

Why Blinn is usually preferred:

- Cheaper
- More stable highlights
- Looks better at grazing angles

So instead of reflection vectors, we compute:

- Light direction
- View direction
- Halfway vector

7. Structuring the lighting terms (clean and explicit)

Your fragment shader now computes lighting in three clearly separated steps:

Ambient

A constant base so nothing is pitch black.

```
vec3 ambient = 0.15 * lightColor;
```

This is artist-tunable — not physically correct, but practical.

Diffuse (Lambertian)

```
float diff = max(dot(norm, lightDir), 0.0);
```

```
vec3 diffuse = diff * lightColor;
```

This is the core “shape revealing” term.

Specular (Blinn)

```
vec3 halfwayDir = normalize(lightDir + viewDir);
```

```
float spec = pow(max(dot(norm, halfwayDir), 0.0), shininess);
```

```
vec3 specular = spec * lightColor * 0.5;
```

Notes:

- shininess controls highlight size
- Scaling specular intensity is common and expected
- Blinn is more forgiving than Phong with low-poly meshes

8. Applying lighting to textured *and* untextured surfaces

This is one of the few things you already handled correctly conceptually.

You:

- Choose a base color (texture or fallback)
- Multiply lighting by that base

This keeps lighting consistent across textured and untextured meshes.

Critically, lighting is **not** baked into the texture — it modulates it.

PCG Zero Values Bug

I have increased the frequency of my PCG to scale to my screen size, however every even generation creates zero coordinated (0, 0). How do I fix this?

0, 0, 0 1, -1.61255, 0 2, 0, 0 3, 1.24856, 0.989052 4, 0, -0 5, 0.914006, -1.06138

Ah, that pattern makes sense now — it's coming from **how you're calling GetNoise** with integer offsets. Let me explain:

Perlin / OpenSimplex 2 behavior

- FastNoiseLite::GetNoise(x, y) produces a smooth continuous value.
- But you are calling it like this:

```
float noiseX = AsteroidNoise.GetNoise(asteroidsSpawned + 0.0f, asteroidsSpawned + 100.0f);
```

```
float noiseY = AsteroidNoise.GetNoise(asteroidsSpawned + 100.0f, asteroidsSpawned + 0.0f);
```

- Notice asteroidsSpawned is **an integer**, increasing by 1 each spawn.
- Noise is sampled at 1.0, 2.0, 3.0... — with higher frequency, integer offsets often hit points where the gradient aligns to produce **exactly zero**. That's why every other spawn is 0.

How to fix

You need to sample the noise at **non-integer coordinates**. A simple trick is to scale your asteroidsSpawned index:

```
float step = 0.3f; // fractional step
```

```
float noiseX = AsteroidNoise.GetNoise(asteroidsSpawned * step, asteroidsSpawned * step + 100.0f);
```

```
float noiseY = AsteroidNoise.GetNoise(asteroidsSpawned * step + 100.0f, asteroidsSpawned * step);
```

- step can be any fractional value (<1.0 usually works well).
- This avoids hitting exact zero-crossings of the noise.

The is slightly lag when asteroids respawn, how can I optimise my asteroid spawning to ensure that it runs efficiently without delay

1. Avoid Repeatedly Loading the Same Model

Currently, each time you spawn an asteroid:

```
asteroids.push_back(Obstacle("textures/Asteroids/Rocky_Asteroid_6.obj", ...));
```

The Obstacle constructor calls:

```
: obstacleModel(path)
```

Which loads the OBJ file into memory. This is extremely expensive if done frequently in a loop. Even though you have a max of 10 asteroids, as the game runs, repeated spawns cause repeated disk/model parsing operations.

Fix: Load the model once and reuse it for all asteroids.

```
// Load model once
```

```
Model asteroidModel("textures/Asteroids/Rocky_Asteroid_6.obj");
```

```
// Pass reference to asteroid
```

```
asteroids.push_back(Obstacle(&asteroidModel, vec3(xPos, yPos, zPos), asteroidSpeed, 0.0f, rotationSpeed, rotationAxis, vec3(1.0f)));
```

You'd need to modify your Obstacle class to accept a Model* or Model& instead of a file path:

```
Obstacle::Obstacle(Model* model, glm::vec3 position, float moveSpeed, ...) : obstacleModel(model), position(position), ...
```

```
}
```

This change alone will massively reduce lag.

2. Pre-allocate Asteroid Objects (Object Pooling)

Instead of dynamically `push_back` every time, you can pre-allocate a fixed pool of obstacles and just reset their positions when needed. This avoids `std::vector` reallocations and repeated object constructions:

```
std::vector<Obstacle> asteroidPool(maxAsteroids);

for (auto& asteroid : asteroidPool) {
    asteroid.resetPosition();
}
```

// When spawning:

```
for (auto& asteroid : asteroidPool) {
    if (!asteroid.isActive()) {
        asteroid.activate(randomPos, rotation, speed);
        break;
    }
}
```

This is much faster than constantly creating/destroying objects and erasing from vectors.

3. Avoid Expensive Collision Box Transformations Every Frame

In your `getCollisionBox` function:

```
glm::mat4 modelMat = getModel();

glm::vec4 worldV = modelMat * glm::vec4(corners[i], 1.0f);
```

You recalculate the transformed bounding box every frame. For simple shapes like asteroids that don't scale or rotate too much, consider:

- Using an axis-aligned bounding sphere instead of a box. This is a simple $\text{distance} < \text{radius}$ check.
- Or precompute rotation-independent bounding boxes in local space, then just translate by position.

This reduces CPU overhead and simplifies collision checks.

4. Avoid Erasing from Vector Every Frame

Currently:

```
if (asteroids[i].getPosition().z > 1.0f) {
    asteroids.erase(asteroids.begin() + i);
    i--;
}
```

erase is $O(n)$ and shifts elements in the vector. With many asteroids, this will introduce noticeable lag.

Alternatives:

- Use a bool active flag instead of erasing:

```
asteroids[i].active = false;
```

- Then skip inactive ones when updating/drawing.
- Only compact the vector occasionally or during level transitions.

Code Optimisation

This is my completed main code for my openGL project, can you analyse and suggest ways in which I can optimise any sections (such as removing redundant code and optimising the game loop) before completion?

1. Game Loop Optimization

Your main loop does a lot of repetitive work:

- You repeatedly call `modelShader.use()`, `setMat4("view", ...)`, `setMat4("projection", ...)` for every object, even though view and projection do not change per object in most frames.
- You also recompute `cameraTilt * lookAt(...)` every frame. While necessary for mouse tilt, you can simplify by computing **once** and reusing for all objects.

Suggestions:

- **Set shader once per frame for common uniforms:**

```
modelShader.use();
```

```
modelShader.setMat4("view", view);
```

```
modelShader.setMat4("projection", projection);
```

```
modelShader.setVec3("viewPos", cameraPosition);
```

```
modelShader.setVec3("lightPos", vec3(-80.0f, 10.0f, -90.0f));
```

Then, only update model or modelColor per object.

- **Minimize use() calls:** You often call modelShader.use() before every object (player, asteroids, planets, signature). Usually, a single call per frame is enough unless you switch shaders.
 - **Star drawing with immediate mode (glBegin/glEnd)** is very slow. Consider **using a VBO/VAO with glDrawArrays** or instancing for all stars. This is a significant performance gain when rendering hundreds of points.
-

2. Asteroid & Star Updates

- In spawnAsteroid() and star updates, you generate random numbers repeatedly. Instead of doing $(\text{rand}() / \text{RAND_MAX}) * 2 - 1$ all the time, you can write a helper function:

```
float randRange(float min, float max) {  
    return min + static_cast<float>(rand()) / RAND_MAX * (max - min);  
}
```

Then:

```
float xPos = randRange(-spawnRangeX, spawnRangeX);
```

```
float yPos = randRange(-spawnRangeY, spawnRangeY);
```

Cleaner, easier to maintain.

- **Asteroid reset logic** is duplicated in the asteroid loop and initial spawn. Consider putting a resetPosition() method in Obstacle.
- **Star noise:** You recompute starNoise.GetNoise for each star every reset. This could be done once at initialization and stored, or precompute some noise patterns if you want static positions.

3. Camera / View Matrix

- You calculate cameraTilt * lookAt(...) per frame. This is fine, but computing rotation matrices per axis with rotate multiple times is a bit heavy. You could precompute a single rotation quaternion from mouse input and convert to matrix—faster and more concise.

4. Repeated Matrix Updates

For planets, asteroids, and the signature, you do:

```
modelShader.setMat4("view", view);  
modelShader.setMat4("projection", projection);  
modelShader.setMat4("model", <objectModel>);
```

You only need to update model per object. view and projection are constant per frame.

- Could refactor:

```
modelShader.use();  
modelShader.setMat4("view", view);  
modelShader.setMat4("projection", projection);  
  
for (auto& planet : planets) {  
    modelShader.setMat4("model", planet.getModelMatrix());  
    planet.Draw(modelShader);  
}
```

5. Lighting

- Light updates (lightGlow) happen every frame. This is fine, but you can calculate the vector once:

```
vec3 lightColor = vec3(3.0f, 3.0f, 2.6f) * (1.0f + 0.2f * sin(timeElapsed));  
modelShader.setVec3("lightColor", lightColor);
```

Then reuse.

- Currently, lightEnabled is toggled per object. If all objects use the same light, set it once per frame unless needed differently.
-

6. Player Input

- processUserInput() is fine but could be slightly cleaner using a small map or array for keys/directions if you plan to expand. Not critical.
-

7. Redundant Variables

- spawnRangeX and spawnRangeY are defined globally and locally multiple times.
- asteroidsSpawned is set to 0.5f but never used.
- program is declared globally but never used.

Cleanup:

- Remove unused variables.
 - Keep a single source of truth for spawn ranges (either global or passed explicitly).
-

8. Rendering Hints

- OpenGL immediate mode (glBegin(GL_POINTS) for stars) is deprecated and slow.
 - Use **VBOs with glBufferSubData** for dynamic star positions, or **instanced rendering**.
 - Consider **frustum culling** for planets and asteroids outside view; saves unnecessary draw calls.
-

9. General Code Organization

- Consider splitting your game loop into smaller functions:
 - updateGame(deltaTime)
 - renderGame()
 - updateStars(deltaTime)
 - updateAsteroids(deltaTime)

This improves readability and reduces repeated code.

10. Minor Optimizations

- `glEnable(GL_CULL_FACE)` can be done once outside the loop.
- `glActiveTexture(GL_TEXTURE0)` could be set once unless changing textures.
- Avoid repeated `glBindVertexArray` if unnecessary.
- For debugging output (`cout << ...`), you can buffer updates instead of printing every frame (performance hit on console).

Planet Calls

Can you update my other planet initializations so that they use the same variables within the call?

Sure! Here's a concise planet initialization:

```
Planet planet1(planet1Model, glm::vec3(-20, 3, -100), glm::vec3(15.0f), glm::vec3(0.0f, 1.0f, 0.0f), glm::radians(5.0f), glm::vec3(0.0f, 0.0f, 0.3f));
```

```
Planet planet2(planet2Model, glm::vec3(15.0f, -4.0f, -80.0f), glm::vec3(8.0f), glm::vec3(0.0f, -1.0f, 0.0f), glm::radians(8.0f), glm::vec3(0.0f, 0.0f, 0.6f));
```

```
Planet planet3(planet3Model, glm::vec3(-80.0f, 10.0f, -90.0f), glm::vec3(40.0f), glm::vec3(0.0f, 1.0f, 0.0f), glm::radians(1.0f), glm::vec3(0.0f, 0.0f, 0.01f));
```

```
Planet planet4(planet4Model, glm::vec3(-50.0f, 10.0f, -140.0f), glm::vec3(20.0f), glm::vec3(1.0f, 0.0f, 0.0f), glm::radians(2.0f), glm::vec3(0.2f, 0.0f, 0.002f));
```

Then in the loop:

```
planet1.updatePosition(deltaTime); planet1.draw(modelShader);
```

```
planet2.updatePosition(deltaTime); planet2.draw(modelShader);
```

```
planet3.updatePosition(deltaTime); planet3.draw(modelShader);
```

```
planet4.updatePosition(deltaTime); planet4.draw(modelShader);
```